



AccoTEST STS8300 应用培训

FOVle 单板硬件及应用

2022-1-11

Make Testing More Valuable

目录

- 单板特性
- 指标
- 原理框图及外观结构
- 脉冲输出功率曲线
- 槽位及对应PogoPin定义
- VI源功能
- AWG功能

- 单板特性

SM8223 八通道浮动电压电流源模块是 STS 8300 系统中的标准模块，模块提供八路精密四象限 VI 源通道，每四个通道一组，两组独立全浮动。

• 单板特性

- 单板八通道分组浮动，每组四通道（每组共用一路 FL 和 SL）
- 全四象限输出电压电流
- 带远端地线补偿的 Kelvin 连接
- 通断检测功能
- 6 个电压量程档：
 - $\pm 40V$, $\pm 20V$, $\pm 10V$, $\pm 5V$, $\pm 2V$, $\pm 1V$
- 6 个电流量程档：
 - $\pm 1A$ (脉冲), $\pm 100mA$, $\pm 10mA$, $\pm 1mA$, $\pm 100\mu A$, $\pm 10\mu A$ （仅测量）
- 电流测量放大能力：
 - X1, X10
- 每通道电压电流可同时测量
- 每通道具备独立电压测量模式 (Relay Sense ON Mode)
- 每通道具备硬件数据处理功能，包括最大值、最小值、平均值及跳变位置
- 每通道具备可编程窗口式电压及电流箝位
- 每通道具备独立的限流保护功能
- 隔离电压 200V
- 支持与 STS 8300 系统内模块同步功能
- 支持多工位并行测试工作模式
- 本板保存校准数据
- 本板自带温度监测功能

• 指标

Voltage Force/Measure

Range	Resolution(16-Bit)	Accuracy
±40V	1.526mV	$\pm(7.5\text{mV} + 0.025\% \text{ Rdg})$
±20V	0.610mV	$\pm(3\text{mV} + 0.025\% \text{ Rdg})$
±10V	0.305mV	$\pm(1.5\text{mV} + 0.025\% \text{ Rdg})$
±5V	0.153mV	$\pm(0.75\text{mV} + 0.025\% \text{ Rdg})$
±2V	0.061mV	$\pm(0.4\text{mV} + 0.025\% \text{ Rdg})$
±1V	0.031mV	$\pm(0.25\text{mV} + 0.025\% \text{ Rdg})$

Current Force

Range	Resolution(16-Bit)	Accuracy
±1A	30.517uA	$\pm(0.2\text{mA} + 0.25\% \text{ Rdg} + 10\text{uA/V})$
±100mA	3.052uA	$\pm(15\text{uA} + 0.05\% \text{ Rdg} + 1\text{uA/V})$
±10mA	305.176nA	$\pm(1.5\text{uA} + 0.05\% \text{ Rdg} + 0.1\text{uA/V})$
±1mA	30.518nA	$\pm(150\text{nA} + 0.05\% \text{ Rdg} + 0.01\text{uA/V})$
±100uA	3.052nA	$\pm(25\text{nA} + 0.05\% \text{ Rdg} + 1\text{nA/V})$

• 指标

Current Measure(X1 amplifier)

Range	Resolution(16-Bit)	Accuracy
±1A	30.517uA	±(0.2mA + 0.25% Rdg + 10uA/V)
±100mA	3.052uA	±(15uA + 0.05% Rdg + 1uA/V)
±10mA	305.176nA	±(1.5uA + 0.05% Rdg + 0.1uA/V)
±1mA	30.518nA	±(150nA + 0.05% Rdg + 0.01uA/V)
±100uA	3.052nA	±(25nA + 0.05% Rdg + 1nA/V)
±10uA	0.305nA	±(10nA + 0.25% Rdg + 0.2nA/V)

Current Measure(X10 amplifier)

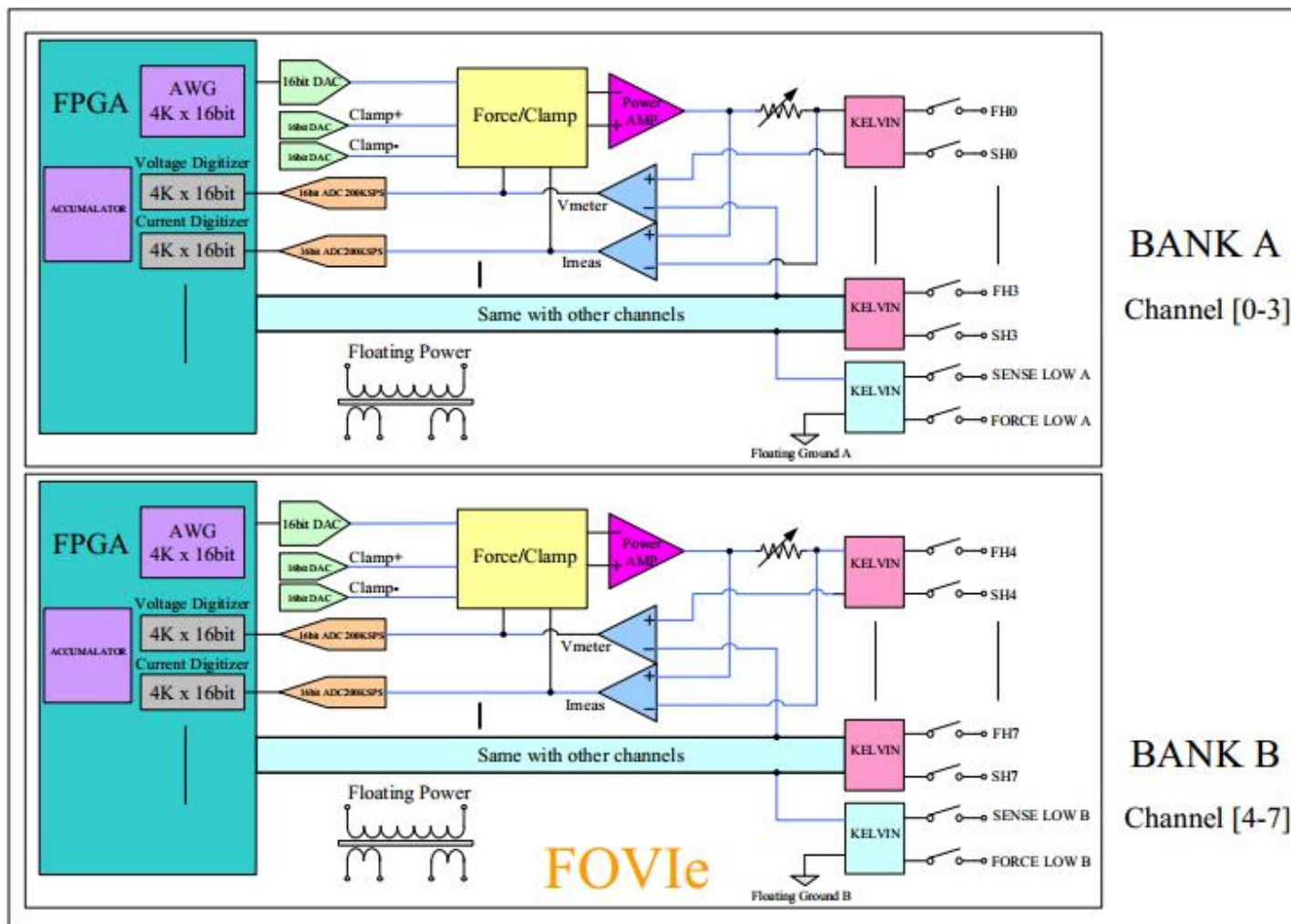
Range	Resolution(16-Bit)	Accuracy
±1A	3.052uA	±(70uA + 0.25% Rdg + 6uA/V)
±100mA	305.176nA	±(7uA + 0.05% Rdg + 0.6uA/V)
±10mA	30.518nA	±(0.7uA + 0.05% Rdg + 60nA/V)
±1mA	3.052nA	±(70nA + 0.05% Rdg + 6nA/V)
±100uA	0.305nA	±(15nA + 0.05% Rdg + 0.8nA/V)
±10uA	0.031nA	±(5nA + 0.25% Rdg + 0.1nA/V)

• 指标

Clamp Voltage Ranges	Resolution	Accuracy (%FS)
$\pm 40\text{V}$, $\pm 20\text{V}$, $\pm 10\text{V}$, $\pm 5\text{V}$, $\pm 2\text{V}$	16bit	$\pm 0.5\%$
$\pm 1\text{V}$	16bit	$\pm 1\%$
Clamp Current Ranges	Resolution	Accuracy (%FS)
$\pm 100\text{mA}$, $\pm 10\text{mA}$, $\pm 1\text{mA}$, $\pm 100\mu\text{A}$, $\pm 10\mu\text{A}$	16bit	$\pm 0.5\%$
$\pm 1\text{A(Pulse)}$	16bit	$\pm 2\%$

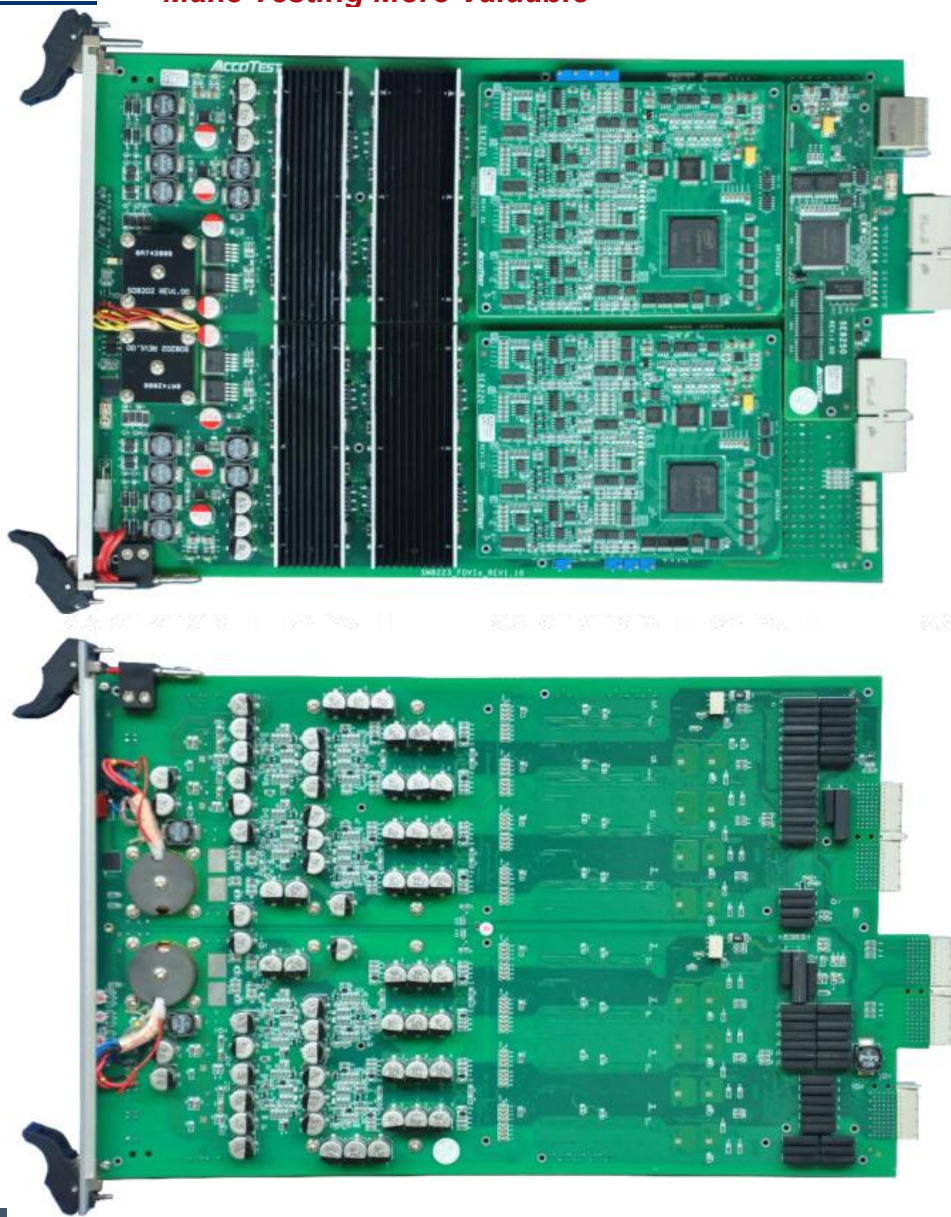
AWG	Resolution	Pattern Depth / Per Channel	DA Update Rate	AWG step
	16bit	4K	100KHz	100us
Digitizer	Resolution	Capture Memory / Per Channel	Maximum Sample Rate	
	16bit	4K	200kSPS	

原理框图



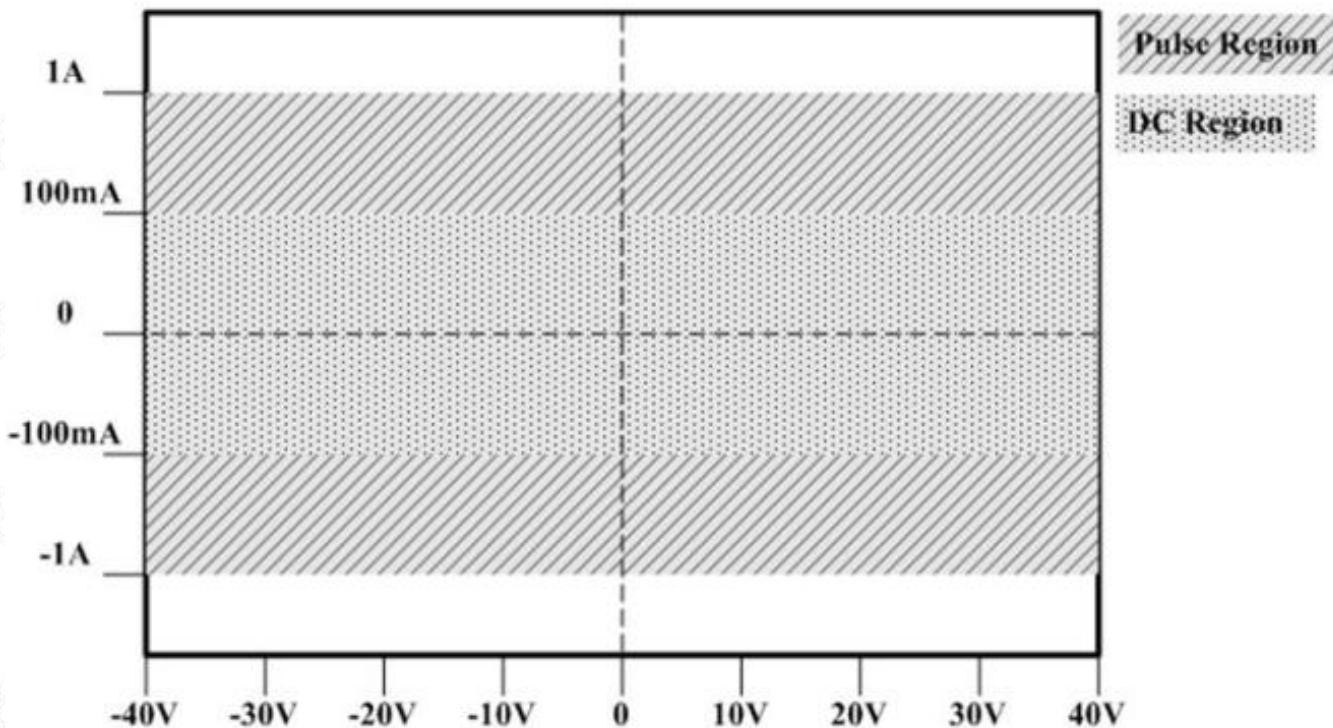
FOVle Block Diagram

- 外观结构



• 四象限输出能力

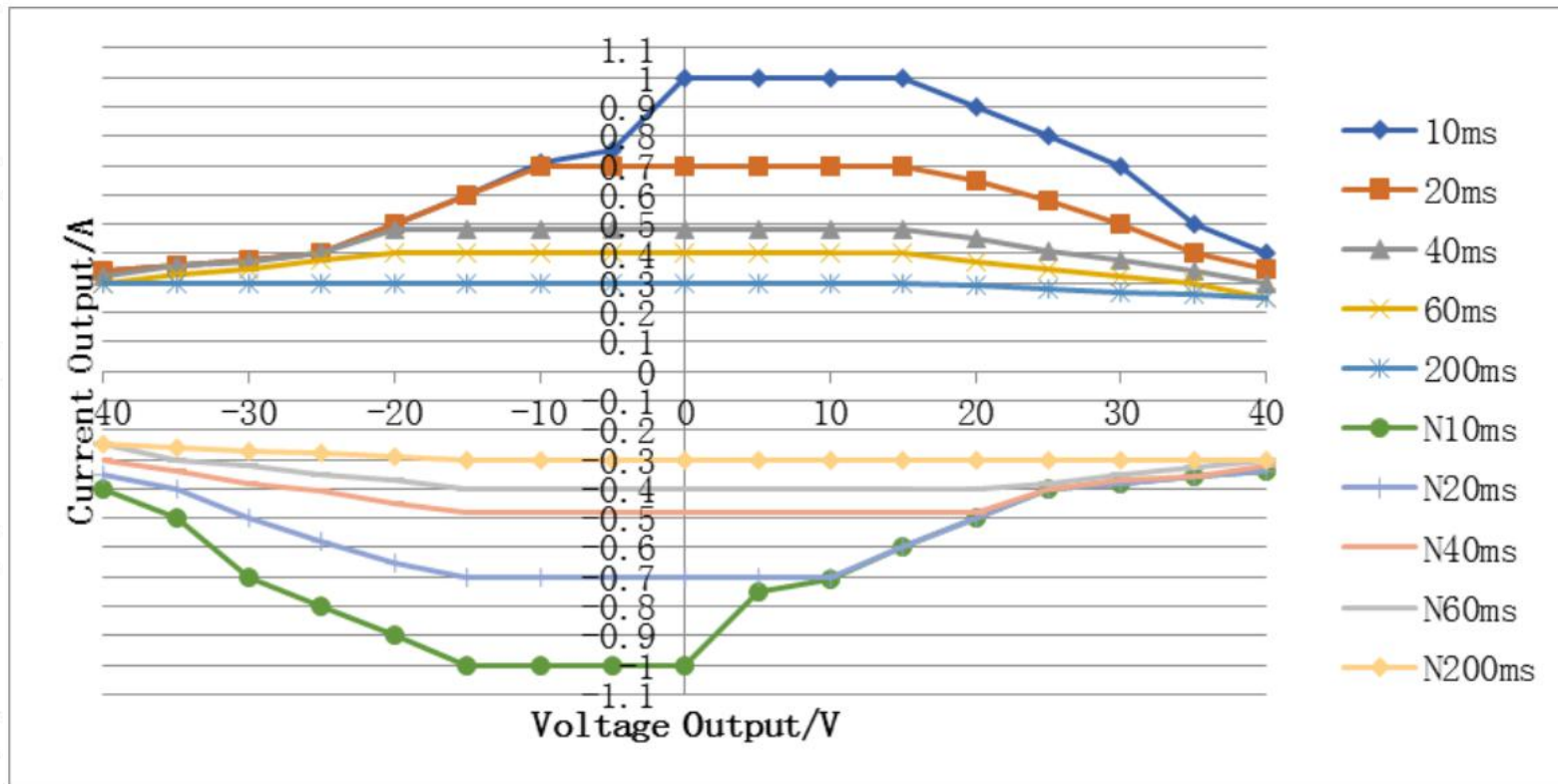
FOVie 每通道在全电压范围内可输出 100mA 直流及 1A 脉冲电流，四象限电压电流输出范围如图所示。



FOVie 四象限电压电流输出范围

脉冲输出功率曲线

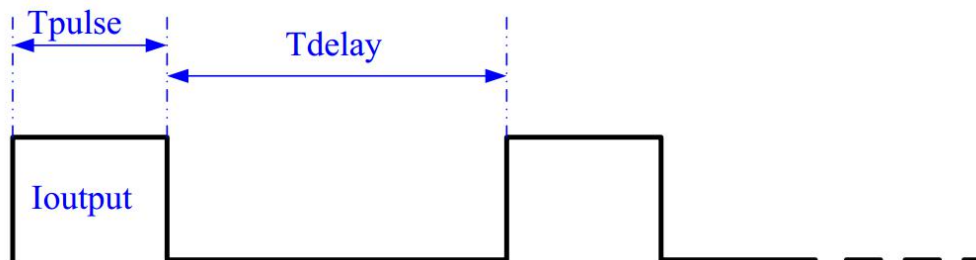
当输出电流大于 100mA 时，在编程时请参考输出功率曲线范围，并正确的调整输出占空比，以确保加在被测器件上的输出电压及输出电流的正确



FOV1e 1A 档脉冲输出功率曲线

• 脉冲输出功率曲线

大电流脉冲工作模式下依靠电容储能，在脉冲大电流后，会依靠 150mA 的恒流源充电，直至电容充满。因此连续大电流输出时，需要合理控制输出占空比以确保电容储能可以恢复。用户在脉冲工作模式下，可以按下式估算脉冲输出后的等待时间：



FOV1e 脉冲工作模式下时间设置

$$T_{delay} = T_{pulse} * I_{output} / 0.15$$

其中：

I_{output} 为脉冲电流值，单位为 A；

T_{pulse} 为电流脉冲持续时间，单位为 s；

T_{delay} 为脉冲输出后电容储能恢复所需的等待时间，单位为 s。

• 脉冲输出功率曲线

大电流脉冲工作模式下依靠电容储能，脉冲电流的持续时间与储能电容容值和输出电压值等因素有关，其近似计算公式如下所示。

$$t = \frac{[48 - (V_{out} + I_{out} * 4.5)] * 300 * 1e - 6}{I_{out} - 0.25}$$

其中 I_{out} 为脉冲电流值，单位为 A；

t 为电流脉冲持续时间，单位为 s；

V_{out} 为输出电压值。

槽位及对应PogoPin定义

F0Vle对应的Analog槽位（TIB类型相关）

33	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	34
NULL	Analog	Analog	Analog	Analog	Analog	Analog	Analog	Analog	DCM	QTMUe	Analog	Analog	Analog	Analog	Analog	Analog	CBITE
	Analog	Analog	Analog	Analog	Analog	Analog	Analog	Analog	DCM	QTMUe	Analog	Analog	Analog	Analog	Analog	Analog	Cal
36	32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	35

A28D4 Config

33	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	34
NULL	Analog	Analog	Analog	Analog	Analog	Analog	TMU	Digital	Digital	TMU	Analog	Analog	Analog	Analog	Analog	Analog	CBite
	Analog	Analog	Analog	Analog	Analog	Analog	TMU	Digital	Digital	TMU	Analog	Analog	Analog	Analog	Analog	Analog	CAL
36	32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	35

A24D8 Config

• 槽位及对应PogoPin定义

From PogoPin Tool

P102_slot2slot1

K	J	I	H	G	F	E	D	C	B	A
			FH_3	FH_2	FL_0-3	FH_1	FH_0	FH_4		
			SH_3	SH_2	FL_0-3	SH_1	SH_0	SH_4	FH_1	FH_0
			G_3	G_2	SL_0-3	G_1	G_0	G_4	SH_1	SH_0
			FH_7	FH_6	FL_4-7	FH_5	FH_4	FH_5	G_1	G_0
			SH_7	SH_6	FL_4-7	SH_5	SH_4	SH_5	FL_4-7	FL_0-3
			G_7	G_6	SL_4-7	G_5	G_4	G_5	FL_4-7	FL_0-3
								FH_6	SL_4-7	SL_0-3
								SH_6	FH_3	FH_2
								G_6	SH_3	SH_2
								FH_7	G_3	G_2
AGND	AGND							SH_7	AGND	AGND
								G_7		

1
2
3
4
5
6
7
8
9
10
11
12

Open Config slot

Resource:

FOVie

Slot:

slot1

Serial NO:

0

☒ AutoSwitch

Config File Path:

C:/AccoTest/AccoTEST System/ConfigEx.slot

Cell Channel And Slot:

P000 P001 P002 P003 P004 P005 P006 P007 P008 P009

P010 P011 P012 P013 P014 P015 P016 P017 P018 P019

P020 P021 P022 P023 P024 P025 P026 P027 P028 P029

P030 P031 P032 P033 P034 P035 P036 P037 P038 P039

- VI源功能

ContackCheck()

GetContackCheck()

Set()

SetSyn()

SetClamp()

MeasureVI()

GetMeasResult()

BlockRead()

Pulse()

• FOVle - 接触电阻检测

ContactCheck (FOVle_CONTACTMODE *checkMode*) ;

Remarks

设置高端接触检测的模式。

注意:

- 1) 通过该函数对高端电路进行接触检测，检测结束后，VI 源将恢复至检测之前的状态；
- 2) 该函数支持 Group 操作。

Parameters

checkMode

接触测试的模式，可设置为：

FOVle_HIGH_SIDE 检测高端电路的连接

```
fovie0.ContactCheck(FOVle_HIGH_SIDE); // set the contact check mode to high side
```

• FOVle -接触电阻检测

GetContactCheckResult (UINT *siteCount*) ;

Remarks

读取该工位对应 Contact 检测模式下的检测结果。

Parameters

siteCount UINT 型变量，选择读取接触检测结果的工位。 0 代表 Site1， 1 代表 Site2.....

Return Values

返回-1: 该工位无效

返回 0: 该工位接触检测 Pass

返回 1: 该工位接触检测 Fail

```
int result = fovie0. GetContactCheckResult(0); // Get contact check result of SITE_1
```

• FOVle

```
Set(VIMode viMode, double setValue, FOVle_VRNG vRange,  
FOVle_IRNG iRange, FOVle_OUT_RELAY relayStatus =  
FOVle_RELAY_HOLD, double risingTime = 0.1);
```

Remarks

设置 FOVle 的状态，包括工作模式、恒压或恒流值、电压量程或电流量程、输出继电器状态及 Capload 时间。

Parameters

viMode FOVle 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-40~40。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-1~1。

vRange FOVle 电压量程。

FOVle_40V, FOVle_20V, FOVle_10V,

FOVle_5V, FOVle_2V, FOVle_1V

iRange FOVle 电流量程。

FOVle_1A, FOVle_100MA, FOVle_10MA,

FOVle_1MA, FOVle_100UA, FOVle_10UA,

注意：

FOVle_10UA 电流档位仅能用于测量。

• FOVle

Parameters

relayStatus 设置输出继电器如何动作，分为七种情况：

FOVle_RELAY_ON：打开输出继电器；

FOVle_RELAY_OFF：关闭输出继电器；

FOVle_RELAY_HOLD：输出继电器保持上一个状态；

FOVle_RELAY_SENSE_ON：启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 FOVle 处于独立电压表模式。

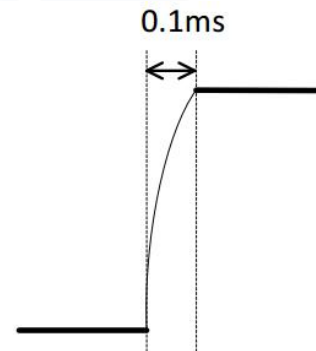
risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.1mS。该参数不写，则默认为 0.1mS；

注意：

- 1) 该参数设置小于等于 0.1mS 时， risingTime=0.1mS，为 Normal 模式；
- 2) 该参数设置小于等于 0.2mS 时， risingTime=0.2mS，为 Slow 模式；
- 3) 该参数设置大于 0.2mS 时，自动启动 Capload 功能， risingTime 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

```
fovie0.Set(FV,5,FOVle_10V,FOVle_100MA,FOVle_RELAY_ON);
```



• FOVle

```
SetSyn(VIMode viMode, const double* setValue, UINT siteSize,
FOVle_VRNG vRange, FOVle_IRNG iRange, FOVle_OUT_RELAY
relayStatus = FOVle_RELAY_HOLD, double risingTime = 0.1);
```

Remarks

对不同的工位设定不同的电压值或电流值，且该操作对不同工位完全同步。

Parameters

viMode FOVle 工作模式，有两种模式：

FV：选择恒压模式

FI：选择恒流模式

setValue 存放每个工位设定的恒压值或者恒流值的数组。

选择恒压模式时，数组中的元素为恒压值，单位为 V。最大范围为-40~40。

选择恒流模式时，数组中的元素为恒流值，单位为 A。最大范围为-1~1。

siteSize UINT 型变量，设置执行同步操作的工位数。

若该数值为 4，则执行同步操作的工位为 SITE1、SITE2、SITE3、SITE4。

- 注意：**
- 1) *setValue* 数组长度必须大于 *siteSize* 的值，否则可能会发生不可预期的错误；
 - 2) 若 *siteSize* 值小于当前有效工位数，则用户未设定的工位状态不做处理，即保持其原来的状态；
 - 3) 若 *siteSize* 值大于当前有效工位数，则按当前有效工位数来处理各工位的状态。

vRange FOVle 电压量程。

FOVle_40V, FOVle_20V, FOVle_10V,

FOVle_5V, FOVle_2V, FOVle_1V

iRange FOVle 电流量程。

FOVle_1A, FOVle_100MA, FOVle_10MA,

FOVle_1MA, FOVle_100UA, FOVle_10UA

注意： FOVle_10UA 电流档位仅能用于测量。

• FOVle

Parameters

relayStatus 设置输出继电器如何动作，分为七种情况：

FOVle_RELAY_ON: 打开输出继电器；

FOVle_RELAY_OFF: 关闭输出继电器；

FOVle_RELAY_HOLD: 输出继电器保持上一个状态；

FOVle_RELAY_SENSE_ON: 启动独立测量模式。只打开 SENSE 线输出继电器，关闭 FORCE 线输出继电器。此时 FOVle 处于独立电压表模式。

risingTime

double 型变量，设置变化时间。单位为 mS，分辨率为 0.1mS。该参数不写，则默认为 0.1mS；

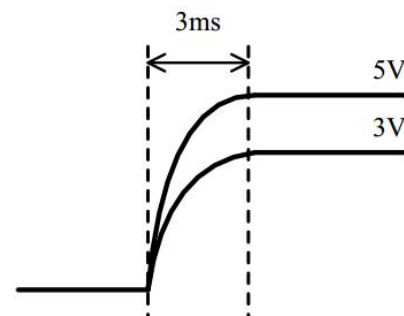
注意：

- 1) 该参数设置小于等于 0.1mS 时， *risingTime*=0.1mS，为 Normal 模式；
- 2) 该参数设置小于等于 0.2mS 时， *risingTime*=0.2mS，为 Slow 模式；
- 3) 该参数设置大于 0.2mS 时，自动启动 Capload 功能， *risingTime* 为 Capload 上升或下降的时间。使用 Capload 功能可以得到更加平滑的上升沿和下降沿，并且上升或者下降时间可以根据需要来设定，Capload 的最大值为 65mS。

• FOVle

例 1: fovie0 通道的 SITE1 设置其输出 3V, SITE2 设置其输出 5V, 且要求 SITE1 和 SITE2 同步输出。同时设置 risingTime 为 3mS。

```
double setValue[2] = {3, 5};
int siteSize = 2;
fovie0.Set(FV,0,FOVle_10V,FOVle_100MA,FOVle_RELAY_ON);
delay_ms(1);
fovie0.SetSyn(FV, setValue, siteSize, FOVle_10V, FOVle_100MA, FOVle_RELAY_ON, 3);
```



• FOVle

SetClamp(double percent_PFS, double percent_NFS);

Remarks

设置 FOVle 的正负箝位值。

注意:

SetClamp 函数与模式相关，切换 FV、FI 模式时会清除箝位设置，箝位值会自动还原成 102%。未切换 FV 和 FI 状态时，箝位设置会保持不变。

Parameters

percent_PFS

输出正箝位值，此处指占正箝位满量程的百分比，单位%。范围： 10~102

percent_NFS

输出负箝位值，此处指占负箝位满量程的百分比，单位%。范围： 10~102

例 1：设置 fovie1 为 FI=0 模式，箝位到(50, 20)，然后切换 fovie1 为 FV=1V 模式，箝位到(25, 25)。

```
fovie1.Set(FI, 0, FOVle_10V, FOVle_10MA, FOVle_RELAY_ON);  
//positive clamp to 50% of full range(FOVle_10V), negative clamp to 20% of full range  
fovie1.SetClamp(50, 20);  
//switch from FI to FV mode, then positive/negative clamps recover back to 102% of full range  
fovie1.Set(FV, 0, FOVle_10V, FOVle_100MA, FOVle_RELAY_ON);  
//reset positive clamp to 25% of full range(FOVle_100MA), negative clamp to 25% of full range  
fovie1.SetClamp (25,25);  
//Set FV=1V  
fovie1.Set(FV,1,FOVle_10V,FOVle_100MA,FOVle_RELAY_ON);
```

• FOVle

```
MeasureVI (UINT sampleTimes, double samplePeriod,
FOVle_MI_GAIN miGain = FOVle_MI_X1, FLOATMEASMODE measMode =
MEAS_NORMAL,UINT T1 = 0,UINT T2 = 0,UINT T3 = 0,UINT T4 = 0);
```

Remarks

设置 FOVle 的测量次数、测量间隔、测流增益、测量模式以及 AWG测量模式下的四次测量启动时刻。

Parameters

sampleTimes 设置 AD 采样次数。范围 1~4096

注意： 当 $sampleTimes < 1$ 时， $sampleTimes = 1$ ；
当 $sampleTimes > 4096$ 时， $sampleTimes = 4096$ 。

samplePeriod 设置每次采样之间的时间间隔，单位uS。最小采样间隔为5uS，分辨率1uS。范围:5~30000。

注意： 当 $samplePeriod < 5$ 时， $samplePeriod = 5$ ；
当 $samplePeriod > 30000$ 时， $samplePeriod = 30000$ 。

miGain FOVle 测流增益。

FOVle_MI_X1, FOVle_MI_X10

该参数可以不写，默认值为 FOVle_MI_X1。

measMode 测量模式选择，可设置为：

MEAS_NORMAL 普通测量模式。调用函数后完成测量。

MEAS_AWG 测量模式。只做设置，运行 AWG 时启动。
该参数可以不写，默认值为 MEAS_NORMAL 模式。

- FOVle

Parameters

$T1$ 、 $T2$ 、 $T3$ 、 $T4$

单位 uS。分辨率 10uS，范围：0 ~ ($T1+T2+T3+T4$) ~ 600000。

AWG 测量模式下，设置测量启动时刻。一次 AWG 中，可以启动四次测量。四个参数均可以不写，默认启动一次，启动时刻为 0。对于这种测量模式，建议跟同步启动测量配合使用。

注意：

$T1$ 、 $T2$ 、 $T3$ 、 $T4$ 为相对时间。即 $T2$ 的启动时间为相对 $T1$ 的启动时间， $T3$ 的启动时间为相对 $T2$ 的启动时间， $T4$ 的启动时间为相对 $T3$ 的启动时间。

例 1：设置 fovie0 为普通模式测量，采样 20 次，间隔 10uS。

```
fovie0.MeasureVI(20,10);
```

• FOVle

```
GetMeasResult(UINT siteCount, MeasRet retType = MVRET,  
int sampleNumber = AVERAGE_RESULT, BYTE triggerNum = 0);
```

Remarks

获取 MeasureVI()函数的测量结果。

1) 如果选择 MVRET 测压模式，则回读的结果为电压值，单位为 V；

2) 如果选择 MIRET 测流模式，则回读的结果为电流值，单位为 A。

注意：

1) FOVle 每次测量时，电压和电流是同时测量的，用户只需要选择读回 MVRET或者 MIRET 即可。

2) 当参数 sampleNumber 选择 TRIG_RESULT 时，如果没有发生触发，则函数返回值为-3，如果测量模式与回读模式不符，则函数返回值为-5。

• FOVle

Parameters

siteCount UINT 型变量，选择测量数据工位。 0 代表 Site1， 1 代表 Site2.....

retype 选择测量读回的数据类型，可设置为

MVRET 回读电压测量数据

MIRET 回读电流测量数据

该参数可以不写，默认值为 MVRET。

sampleNumber

AVERAGE_RESULT 平均值

MAX_RESULT 最大值

MIN_RESULT 最小值

TRIG_RESULT 触发值。AWG 同步测量时，跳变触发的位置。即 MeasureVI 中第几个采样点发生跳变，最多支持两次触发。

非负值 N： 返回的是第 N+1 个采样点的测量值。假设 MeasureVI 的采样次数为 sam，则 $0 \leq N \leq \text{sam} - 1$ 。

该参数可以不写，默认值为 AVERAGE_RESULT。

triggerNum 触发次数设置。

仅当 *sampleNumber* 选择 TRIG_RESULT 时，该参数有效，最多支持两次触发。

该参数可以不写，默认值为 0，表示回读第一次发生触发的位置。该参数设置为 1，表示回读第二次发生触发的位置

- FOVle

例1：返回 fovie0 电流测量结果平均值。

```
//fovie0 sampleTimes=20, samplePeriod=10uS
fovie0.MeasureVI(20, 10);
for(i=0; i<SITENUM; i++)
{
    // the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is average value of current
    adresult[i]=fovie0.GetMeasResult(i, MIRET);
}
```

例2：返回 fovie0 电压测量结果第 15 个点的值。

```
//fovie0 sampleTimes=20, samplePeriod=10uS
fovie0.MeasureVI(20, 10);
for(i=0; i<SITENUM; i++)
{
    // the result is fifteenth voltage value
    adresult[i]=fovie0.GetMeasResult(i, MVRET, 14);
}
```

- FOVle

例 3: AWG 同步测量时, 返回 fovie0 两次跳变触发的位置。

```
// the trigger value is 2V, falling edge
fovie0.SetMeasVTrig(2, TRIG_FALLING);
// fovie1 sampleTime=50, samplePeriod=20uS, miGain is X1, measure mode is MEAS_AWG
fovie1.MeasureVI(50, 20, FOVle_MI_X1, MEAS_AWG);
// the fovie0's parameters of MeasureVI function are same with fovie1
fovie0.MeasureVI(50, 20, FOVle_MI_X1, MEAS_AWG);
// fovie1 AWG enable
STSEnableAWG(&fovie1);
// fovie0 and fovie1 measure enable
STSEnableMeas(&fovie1, &fovie0);
STSAWGRun();
for(i=0; i<SITENUM; i++)
{
    // Get the voltage position of first trigger
    Trig_Point0[i] = fovie0.GetMeasResult(0, MVRET, TRIG_RESULT, 0);
    // Get the voltage position of second trigger
    Trig_Point1[i] = fovie0.GetMeasResult(0, MVRET, TRIG_RESULT, 1);
}
```

• FOVle

```
BlockRead(UINT siteCount,UINT start,UINT size, double* buffer,  
MeasRet retType = MVRET);
```

Remarks

此函数读取一个数据块的测量结果，数据块的起始地址和大小由用户定义。

Parameters

siteCount UINT 型变量，选择测量数据工位。 0 代表 Site1， 1 代表 Site2.....

start UINT 型变量，数据块的起始地址，非负整数。参数范围：0~（MeasureVI 函数中设置的采样次数-1）

size UINT 型变量，数据块的大小，正整数。参数范围：1~（MeasureVI 函数中设置的采样次数-*start*）

buffer 存放数据结果的数组，由用户定义，如：double result[1000]={0.0};

注意：

- 1) *buffer* 数组长度需不小于 *size* 的值，否则可能会发生不可预期的错误；
- 2) 建议把该数组定义成全局变量。

retype 选择测量读回的数据类型，默认为 MVRET。

MVRET 读回电压测量数据

MIRET 读回电流测量数据

```
double result[SITENUM][200]={0.0};  
fovie0.MeasureVI(200, 10);  
for(i=0; i<SITENUM; i++){  
    //start=0; size=200; Get MVRET, saved to result.  
    fovie0.BlockRead(i, 0, 200, result[i], MVRET);  
}
```


• FOVle

```
Pulse(double pulseValue, UINTpulseTime = 300, int measStartTime  
= 0, UINTsampleTimes= 10, double sampleInterval= 30);
```

Remarks

输出幅度、脉宽可调的电压或电流脉冲。

注意:

Pulse 函数之前，需先调用 Set 函数来设置量程等信息。 Pulse 脉冲的起始值和终止值均为 Set 语句设置值。

Parameters

pulseValue double 型变量，设置脉冲幅值。

选择 FV 模式时，单位 V。最大范围为-40~40；

选择 FI 模式时，单位 A。最大范围为-1~1。

pulseTime UINT 型变量，设置脉冲时间。单位 uS。分辨率： 10uS。范围： 300~40000。

可以不写，默认为 300uS。

measStartTime int 型变量，设置启动测量时间，单位 uS。范围： -60000~60000。可以不写，默认为 0。

sampleTimes UINT 型变量，设置采样次数。范围 1~4096。可以不写，默认为 10。

sampleInterval UINT 型变量，设置采样间隔时间，单位 uS。最小采样间隔为 5uS，分辨率 1uS。

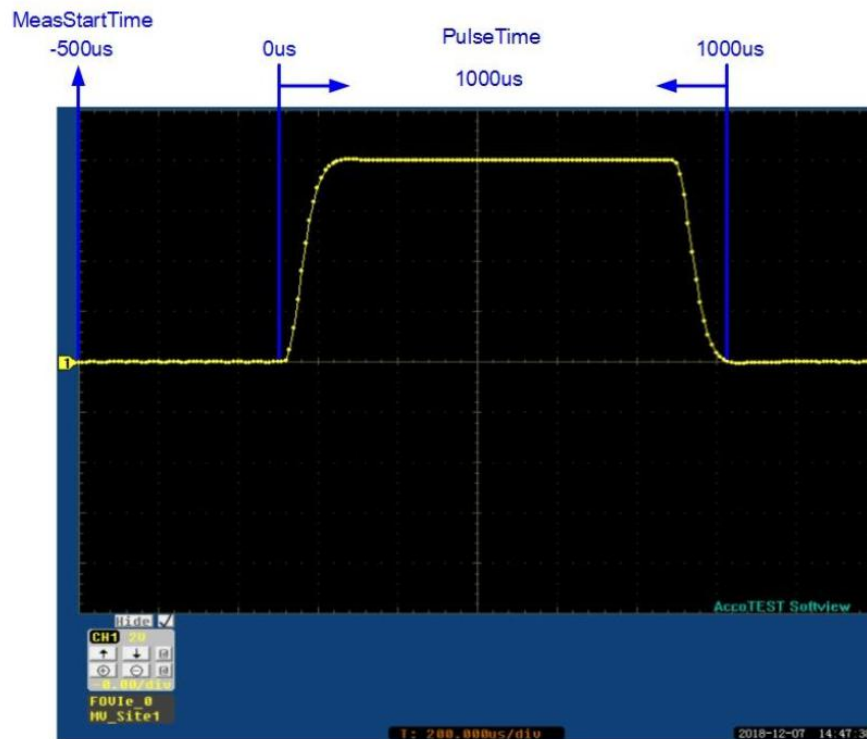
可以不写，默认为 30uS。

• FOVle

例 1: 设置 fovie0 起始状态 FV 为 0。设置 fovie0 输出 1mS 的 8V 脉冲；采样次数200 次，采样间隔时间 10uS，脉冲生成前 500uS 开始启动测量。

```
fovie0 .Set(FV,0,FOVle_10V,FOVle_10MA,FOVle_RELAY_ON);
// first set fovie0 in FV=0 mode,
fovie0.Pulse (8,1000, -500, 200, 10);
//pulse output value=8v, pulse duration=1000uS, start time for measuring is -500uS
//sample=200,interval time=10uS
```

使用 SoftView 软件抓取的波形右图：



- FOVle –AWG

AwgLoader()

AwgSelect()

AwgRun()

AwgStop()

AwgClear()

SetMeasVTrig()

SetMeasITrig()

• FOVle –AWG

```
AwgLoader(char* awgName, VIMode viMode, FOVle_VRNG vRange,  
FOVle_IRNG iRange, double* awgData, UINT awgSize);
```

Remarks

导入 AWG 波形数据。设置导入的 AWG 波形的名称、导入数据的长度及档位信息。

注意:

- 1) 档位信息不涉及档位的切换，仅用于波形数据换算；
- 2) 同一路 FOVle 通道可以导入多个 AWG 波形，但不同 AWG 波形的命名必须不同。对于相同名称的 AWG 波形，只写入第一次调用 AwgLoader 函数中设置的波形；
- 3) 启动 AwgLoader 函数会把本次的 AWG 波形写入 RAM 中，因此执行该语句会占用测试时间。一次性写入 4096 个点的数组，执行 AwgLoader 函数需要 5mS 左右的时间，写入时间与写入点数成正比；
- 4) 同一个测试程序中，同一路 FOVle 通道每次导入的 AWG 波形总长度必须小于等于 4096，超出 4096 以后的数据不会写入 RAM；
- 5) 同一个测试程序中，建议同一路 FOVle 通道导入的多个 AWG 波形总长度小于4096。此时，同一个 AWG 波形数据不会被重复导入，仅第一次执行 AwgLoader 函数时占用测试时间；
- 6) 同一个测试程序中，如果同一路 FOVle 通道导入的多个 AWG 波形总长度大于4096。此时，每次会重新写入新的 AWG 数据到 RAM 中，增加了测试时间。

• FOVle –AWG

Parameters

awgName char 型变量, AWG 波形名称。命名不区分大小写。

viMode FOVle 工作模式, 有两种模式:

FV 选择恒压模式

FI 选择恒流模式

vRange FOVle 电压量程。

FOVle_40V, FOVle_20V, FOVle_10V

FOVle_5V, FOVle_2V, FOVle_1V

iRange FOVle 电流量程。

FOVle_1A, FOVle_100MA, FOVle_10MA,

FOVle_1MA, FOVle_100UA, FOVle_10UA

awgData double 型指针, 存储 AWG 波形数据。

awgSize UINT 型变量, 设置 AWG 波形数据的大小。该参数范围为 1~4096。

例1: 建立一个波形数据组, fovie0 从 16.5V 上升到 21.5V, 步距 0.1V。

```
int sam = 50; //AWG waveform data length
int interval = 200; //AWGdata interval time, unit is uS
double awg_pattern[100]={0.0};
STSAWGCreateRampData(&awg_pattern[0],sam,1,16.5,21.5);
//pattern array saved in 'awg_pattern', start from 16.5 to 21.5
fovie0.AwgLoader("Vst ",FV,FOVle_40V,FOVle_100MA,awg_pattern,sam);
```

- FOVle –AWG

例 2：建立一个波形数据组， fovie0 输出频率为 100Hz、峰峰值为 4V、直流偏置为 10V 的正弦波。

```
int sam = 1000; //data size for AWG pattern
int interval = 10; //interval time for AWG pattern
double awg_pattern[2000]={0.0};
STSAWGCreateSineData(&awg_pattern[0],sam,1,4,10,0);
//set a sinewave data array of 100Hz, Vpp=4, DC Offset=10
fovie0.AwgLoader("Sine",FV,FOVle_10V,FOVle_1MA,awg_pattern, sam);
// pattern named as 'SINE', FV mode
```


• FOVle –AWG

```
AwgSelect(char* awgName, UINT startAddr, UINT stopAddr,  
int loopBackAddr, double awgInterval= 10.0);
```

Remarks

选择将要运行的 AWG 波形，同时设置该 AWG 波形的起始点、结束点、回转点以及 AWG 启动间隔时间。

注意：

- 1) 同一个测试程序中每一路 FOVle 通道可以导入多个 AWG 波形，每个 AWG 波形必须有自己的独立名称，通过 AwgSelect 函数来选择将要运行的 AWG 波形。
- 2) AwgSelect 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- 3) AwgSelect 函数需配合 AWG 同步启动函数使用。执行 AwgSelect 并没有 AWG 输出，只有运行同步启动函数 STSAWGRUN 才会有 AWG 波形输出。

Parameters

awgName char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr UINT 型变量，设置 AWG 波形起始点。数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr UINT 型变量，设置 AWG 波形结束点。数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr int 型变量，设置 AWG 波形回转点，即 AWG 执行结束后停止的位置。

当该值小于 0 时，AWG 波形保持在运行结束的位置；

当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。

数据范围为 10~60000。默认值为 10uS。

• FOVle –AWG

例 1：建立两个波形数据组，分别为 AWG_Vst 和 AWG_Vuvlo。仅运行 AWG_Vst波形。

```
double interval = 200; // interval time for AWG pattern, unit is uS
int sam1 = 1000; //data size for AWG pattern
int sam2 = 500; // data size for AWG pattern
double awg_pattern1[1000]={0.0}; //define AWG_Vst data array
double awg_pattern2[500]={0.0}; //define AWG_Vuvlo data array
//set waveform data array for AWG_Vst
STSAWGCreateRampData(&awg_pattern1[0],sam1,1,16.5,21.5);
//set AWG_Vuvlo waveform data array
STSAWGCreateRampData(&awg_pattern2[0],sam2,1,16.5,10.5);
// pattern named as 'AWG_Vst', FV mode
fovie0.AwgLoader("AWG_Vst",FV,FOVle_40V,FOVle_100MA,awg_pattern1,sam1);
// pattern named as 'AWG_Vuvlo', FV mode
fovie0.AwgLoader("AWG_Vuvlo",FV,FOVle_20V,FOVle_100MA,awg_pattern2,sam2);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'AWG_Vst' of its 'AwgLoader()'
fovie0.Set(FV, 16.5, FOVle_40V, FOVle_100MA, FOVle_RELAY_ON);
//Select pattern of 'AWG_Vst', start step=0, stop=sam-1; loop=0; interval time=200uS
fovie0.AwgSelect("AWG_Vst", 0, sam1-1, 0, interval);
.....
STSAWGRun(); //AWG synchronous running started for pattern of 'AWG_Vst'
```

• FOVle –AWG

```
AwgRun(char* awgName, UINT startAddr, UINT stopAddr,  
int loopBackAddr, double awgInterval= 10.0,  
BOOL runMode = AWG_SINGLE, UINT delayTime_ms = 0);
```

Remarks

某一路 FOVle 通道 AWG 单独启动函数。

注意:

- 1) AwgRun 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的AwgLoader 中的量程档位一致。
- 2) 此函数设置了波形的起始点、结束点、回转点以及时间间隔，不需要配合AwgSelect 函数使用。
- 3) 使用该函数不能与其它源同步测量，需要同步测量时，请使用 AwgSelect 配合同步函数 STSAWGRun。

• FOVle –AWG

Parameters

awgName char 型变量， AWG 波形名称。应与之前通过 *AwgLoader* 函数导入的 AWG 波形名称保持一致。

startAddr UINT 型变量，设置 AWG 波形起始点。数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr UINT 型变量，设置 AWG 波形结束点。数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr UINT 型变量，设置 AWG 波形回转点。

当该值小于 0 时， AWG 波形保持在运行结束的位置；

当该值在 0~sam-1 时， AWG 波形运行结束后，会保持在设定点的位置。

awgInterval double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。数据范围为10~60000。

该参数可以不写，默认 10uS。

runMode 运行模式选择，可以设定为：

AWG_SINGLE 运行一次 AWG 波形。

AWG_LOOP 循环运行 AWG 波形，使用 *AwgStop()*函数停止。

默认值为 **AWG_SINGLE**。

delayTime_ms UINT 型变量，单位 mS。范围： 0~60000。设置延时时间。

AWG 输出*delayTime_ms* 之后，系统允许*AwgRun* 之后的操作开始运行。

delayTime_ms 不填的时候，*delayTime_ms* 默认为AWG 扫描时间，只有在AWG 扫描完毕之后，系统才允许*AwgRun* 之后的操作开始运行。

注意：

建议不填该参数！

• FOVle –AWG

例 1: fovie0 连续输出 120Hz 正弦波, 幅值 0.33V, 直流偏置为 3.3V。

```
int sam = 830; //size of AWG pattern
double interval = 10; //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};
// set waveform data array for Sin_AWG
STSAWGCreateSineData(&awg_pattern[0],sam,1,0.66,3.3,0);
// pattern named as 'Sin_AWG, FV mode
fovie0.AwgLoader("Sin_AWG",FV, FOVle_5V, FOVle_1A,awg_pattern,sam);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its 'AwgLoader()'
fovie0.Set(FV, 0, FOVle_5V, FOVle_1A, FOVle_RELAY_ON);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
fovie0.AwgRun("Sin_AWG ",0,(sam-1),0, interval,AWG_LOOP);
```

• FOVle –AWG

AwgStop(void);

Remarks

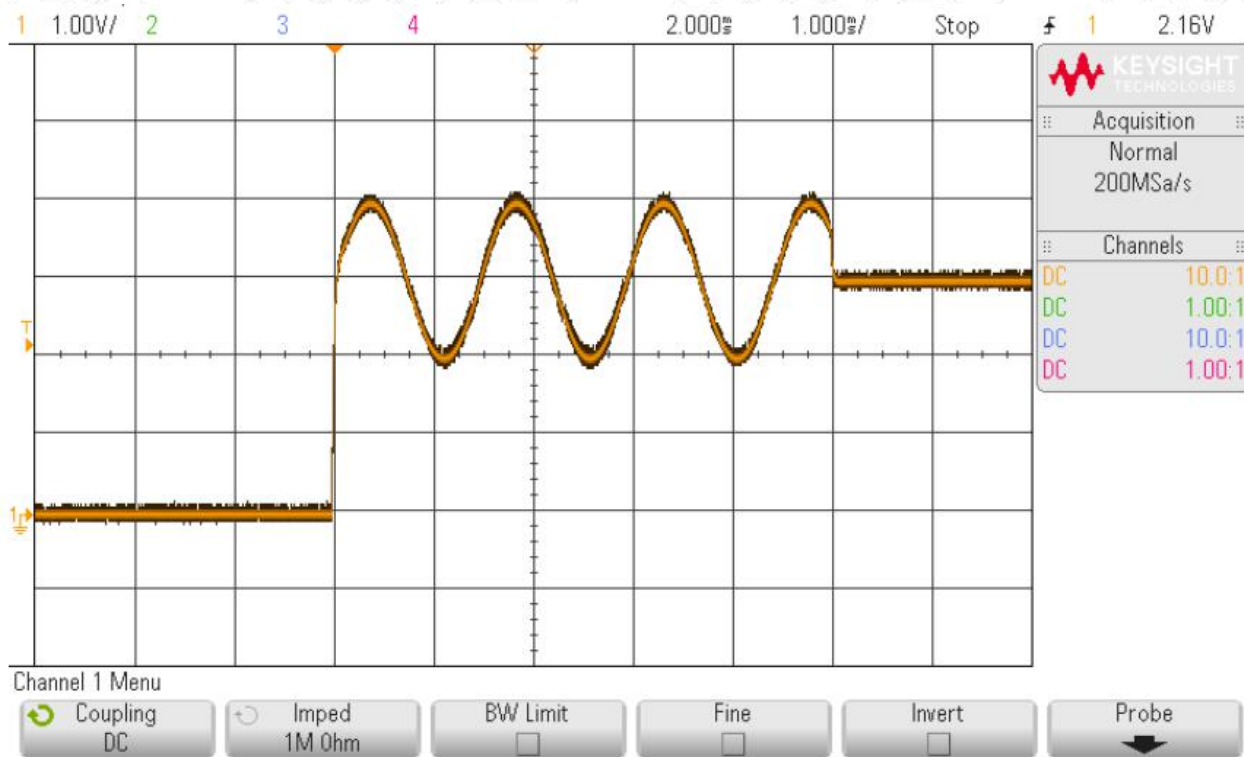
停止某一路 FOVle 通道的 AWG 循环运行。

例 1: fovie0 生成一个周期为 1.5mS 的正弦波，循环运行该正弦波， 5mS 之后停止运行。

```
int sam = 150; //data size of AWG pattern
double interval = 10; //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};
STSAWGCreateSineData(&awg_pattern[0],sam,1,2,3,0);
//Set a Sine wave data array, data size is 150, period is 1, Vpp=2V, DC offset = 3V, phase = 0.
fovie0.AwgLoader("Sin_AWG", FV, FOVle_5V, FOVle_1A, awg_pattern, sam);
//Import awg_pattern data array into fovie0 channel, pattern named as 'Sin_AWG'
fovie0.Set(FV, 0, FOVle_5V, FOVle_1A, FOVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its 'AwgLoader()'
fovie0.AwgRun("Sin_AWG", 0, (sam-1), 0, interval, AWG_LOOP);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS, loop mode
delay_ms(6); //fovie0 AWG pattern keeps running for 6mS
fovie0.AwgStop();
//stop loops for fovie0 AWG running
```


• FOVle –AWG

该正弦波周期为 1.5mS，延时 5mS 时，实际运行了 4 个周期的 AWG 波形。下图为示波器抓取的实时波形。



- FOVle –AWG

AwgClear(void);

Remarks

一般用于 AwgLoader 函数之前，清除该 FOVle 通道的 AWG 数据。

例1： Load 波形之前清除 fovie0 通道的 AWG 数据。

```
fovie0.AwgClear(); //clear AWG data of fovie0  
fovie0.AwgLoader("P1",FV,FOVle_40V,FOVle_100MA,awg_pattern1,sam);
```

• FOVle –AWG

```
SetMeasVTrig(double vTrig, TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

Remarks

电压测量触发设置。

注意:

SetMeasVTrig 函数与 MeasureVI 函数运行时，对应的电压电流档位、测量增益状态必须一致。二者之间不允许插入改变电压电流量程档位及测量增益状态的函数。

Parameters

vTrig double 型变量，设置触发电压。单位 V。

注意:

该参数的值与其对应通道所调用的 Set() 函数的电压档位相关。

trigMode 触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

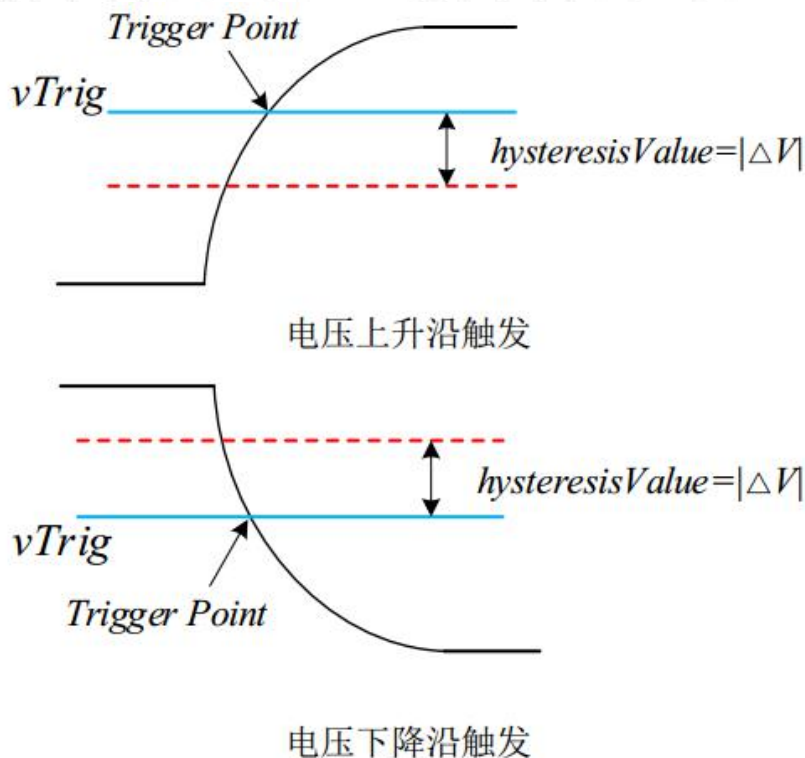
该参数可以不写，默认为 **TRIG_FALLING** 下降沿触发。

hysteresisValue double 型变量，设置触发电压的滞回值。单位 V。在默认条件下等于 0。

• FOVle –AWG

注意:

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值;
- 2) 在上升沿(TRIG_RISING)触发的情况下, 当判定到测量结果先小于($vTrig - hysteresisValue$) 值之后又大于 $vTrig$ 值时, 此时在 $vTrig$ 处判定为触发;
- 3) 在下降沿(TRIG_FALLING)触发的情况下, 当判定到测量结果先大于($vTrig + hysteresisValue$)值之后又小于 $vTrig$ 值时, 此时在 $vTrig$ 处判定为触发。



• FOVle –AWG

例 1：建立一个波形数据组， fovie0 从 10.5V 下降到 8V，步距 0.1V。设置 fovie1下降沿触发，触发电平 2V。

```
fovie1.Set(FI, 0, FOVle_10V, FOVle_10MA, FOVle_RELAY_ON);
```

```
double Trig = 2; //triggering voltage
```

```
double Trig_Point[4]={0.0};
```

```
double result[4]={0.0};
```

```
int sam = 25; //data size of AWG pattern
```

```
double interval = 200; //interval time of AWG pattern, unit is uS
```

```
double awg_pattern[100]={0.0};
```

```
STSAWGCreateRampData(&awg_pattern[0],sam,1,10.5,8);
```

```
//Set a ramp wave data array, data size is 25, period is 1, from 10.5 to 8.
```

```
fovie0.AwgLoader("Uvlo",FV,FOVle_40V,FOVle_100MA,awg_pattern,sam);
```

```
//import the awg_pattern array into fovie0 channel, named it as 'Uvlo'. Set to FV mode.
```

```
fovie0.Set(FV, 10.5, FOVle_40V, FOVle_100MA, FOVle_RELAY_ON);
```

```
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Uvlo' of its 'AwgLoader()'
```

```
fovie0.AwgSelect("Uvlo", 0, sam-1, sam-1, interval);
```

```
//Select pattern of 'Uvlo', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
```

```
fovie1.Set(FI, 0, FOVle_10V, FOVle_10MA, FOVle_RELAY_ON); //set fovie1 on FI mode.
```

```
fovie1.SetMeasVTrig(Trig, TRIG_FALLING); //trigger value is 2V, falling edge
```

```
fovie0.MeasureVI(sam,interval,FOVle_MI_X1, MEAS_AWG);
```

```
fovie1.MeasureVI(sam,interval,FOVle_MI_X1, MEAS_AWG);
```

```
STSEnableAWG(&fovie0); //enable AWG pattern for fovie0
```

```
STSEnableMeas(&fovie0, &fovie1); //enable measurement for fovie0 and fovie1
```

```
STSAWGRun(); //Enable AWG and measurement synchronously
```

SetMeasVTrig/SetMeasITrig函数与MeasureVI函数运行时，对应的电压电流档位状态必须一致。二者之间不允许插入改变电压电流量程档位及增益的函数。

• FOVle –AWG

```
SetMeasITrig(double iTrig, TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

Remarks

电流测量触发设置。

注意:

SetMeasITrig 函数与 MeasureVI 函数运行时，对应的电压电流档位、测量增益状态必须一致。二者之间不允许插入改变电压电流量程档位及测量增益状态的函数。

Parameters

iTrig double 型变量，设置触发电流。单位 A。

注意:

该参数的值与其对应通道所调用的 Set() 函数的电流档位相关。

trigMode 触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

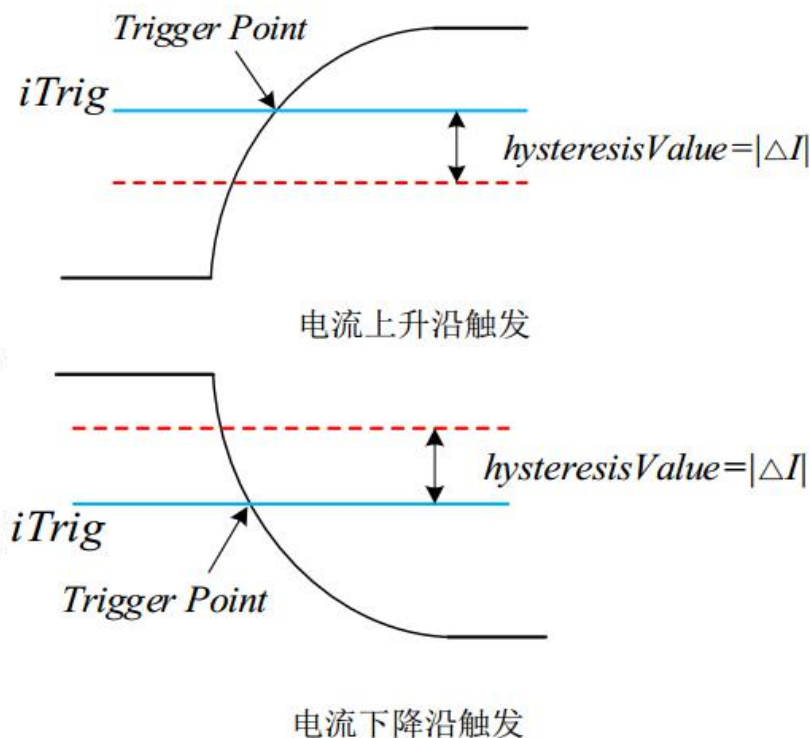
该参数可以不写，默认为 TRIG_FALLING 下降沿触发。

hysteresisValue double 型变量，设置触发电流的滞回值。单位 A。在默认条件下等于 0。

• FOVle –AWG

注意:

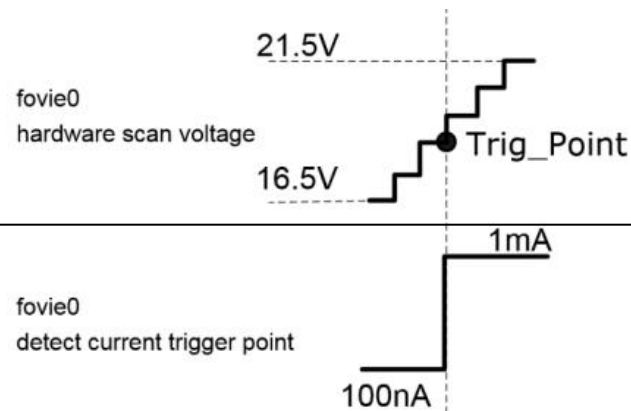
- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值;
- 2) 在上升沿(TRIG_RISING)触发的情况下, 当判定到测量结果先小于($iTrig - hysteresisValue$) 值之后又大于*iTrig* 值时, 此时在*iTrig* 处判定为触发;
- 3) 在下降沿(TRIG_FALLING)触发的情况下, 当判定到测量结果先大于($iTrig + hysteresisValue$)值之后又小于*iTrig* 值时, 此时在*iTrig* 处判定为触发。



• FOVle –AWG&Digitizer 样例

例1: 建立一个波形数据组, fovie0 从 16.5V 上升到 21.5V, 步距 0.1V。测试 fovie0 电流大于 200uA 时的跳变点的电压值。

```
double Trig = 200e-6; // trigger current level
double Trig_Point[4] = {0.0};
double result[4] = {0.0};
int sam = 50; // data size of AWG pattern
int interval = 200; // interval time of AWG pattern, unit is uS
double awg_pattern[100] = {0.0};
STSAWGCreateRampData(&awg_pattern[0], sam, 1, 16.5, 21.5);
// Set an ramp data array, the start address starts from 0, data size is 50.
fovie0.AwgLoader("Vst", FV, FOVle_40V, FOVle_100MA, awg_pattern, sam);
// import the awg_pattern array into fovie0 channel, named it as 'Vst'. Set to FV mode.
fovie0.Set(FV, 16.5, FOVle_40V, FOVle_100MA, FOVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vst' of its 'AwgLoader()'
fovie0.AwgSelect("Vst", 0, sam-1, sam-1, interval);
// Select pattern of 'Vst', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
fovie0.MeasureVI(sam, interval, FOVle_MI_X1, MEAS_AWG);
fovie0.SetMeasITrig(Trig, TRIG_RISING); // trigger value is 200uA, rising edge
STSEnableAWG(&fovie0); // enable AWG pattern for fovie0
STSEnableMeas(&fovie0); // enable measurement for fovie0
STSAWGRun(); // Enable AWG and measurement synchronously
for(i=0; i<SITENUM; i++){
    Trig_Point[i] = fovie0.GetMeasResult(i, MIRET, TRIG_RESULT); // Get the position of trigger point
    result[i] = fovie0.GetMeasResult(i, MVRET, Trig_Point[i]); // Read the voltage value on trigger position
    Vst->SetTestResult(i, 0, result[i]);
}
```



• FOVle –AWG&Digitizer 样例

测试传输延时。设置VCC（fovie0）从0上升到5V，同步测量VCC（fovie0）和Reset（fovie1）的波形，查找Reset端电压Trigger跳变高于2V的跳变点Trig_Point，则（Trig_Point-1）*Interval_Meas即为传输延时时间。

```
int sam = 2; // data size of AWG pattern
int interval = 200; //interval time of AWG pattern, unit is 'uS'
int interval_Meas = 200; // interval time for measurement
double awg_pattern[100]={0.0};
double Trig = 2; //Trigger level
double Trig_Point[4]={0};
awg_pattern[0]=0;
awg_pattern[sam-1]=5;

fovie1.Set(FI, 0, FOVle_10V, FOVle_1MA, FOVle_RELAY_ON); //Reset FI mode, prepare to measure
delay_ms(1);

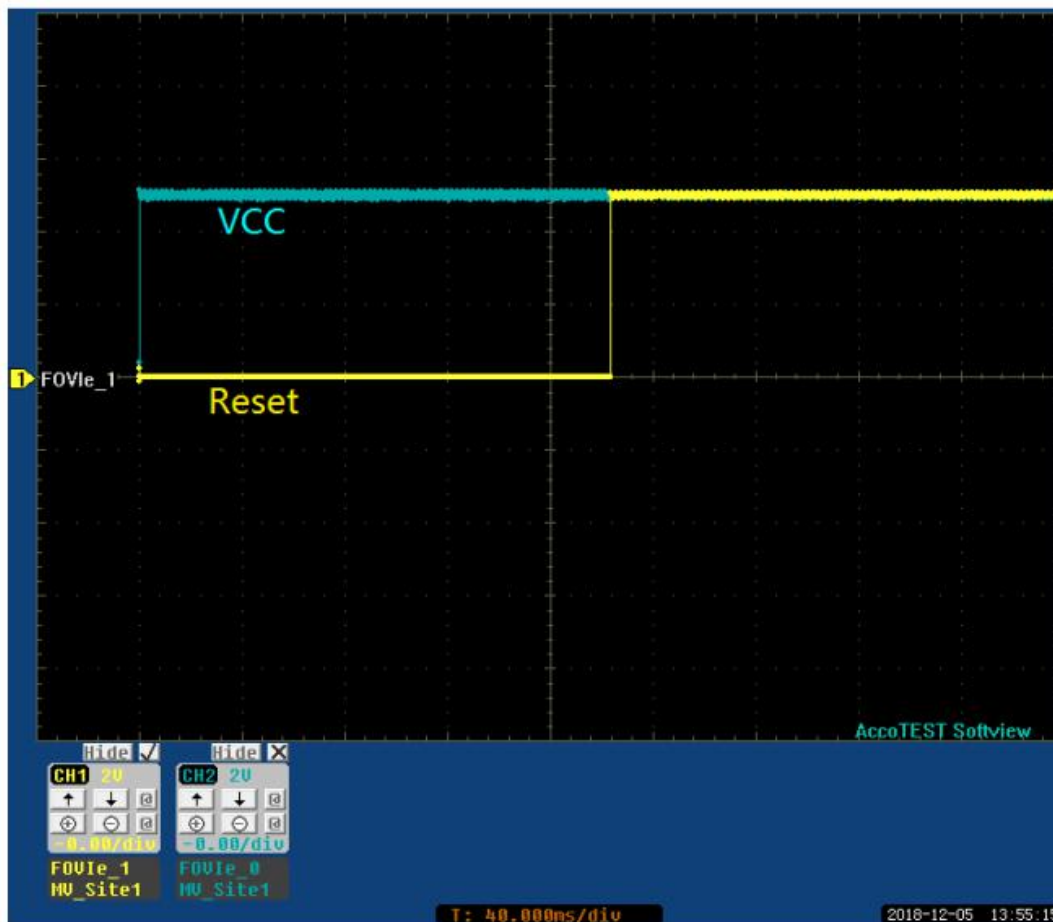
fovie0.AwgLoader("VCC_AWG",FV,FOVle_10V,FOVle_100MA,awg_pattern,sam);
//load the data of "awg_pattern"(sam=2), named "VCC_AWG"
fovie0.Set(FV, 0, FOVle_10V, FOVle_100MA, FOVle_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'VCC_AWG' of its 'AwgLoader()'
fovie0.AwgSelect("VCC_AWG", 0, sam-1, sam-1, interval);
//Select pattern of 'VCC_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
```

• FOVle –AWG&Digitizer 样例

```
fovie1.SetMeasVTrig(Trig,TRIG_RISING); //trigger value is 2V, Rising edge
fovie0.MeasureVI(2000, interval, FOVle_MI_X1, MEAS_AWG);
//fovie0 AWG Mode, SampleTimes=2000, samplePeriod=200uS.
fovie1.MeasureVI(2000, interval, FOVle_MI_X1, MEAS_AWG);
//fovie1 AWG Mode, SampleTimes=2000, samplePeriod=200uS.

STSEnableAWG(&fovie0); //enable AWG pattern for fovie0
STSEnableMeas(&fovie0,&fovie1); //enable measurement for fovie0 and fovie1
STSAWGRun(); //Enable AWG and measurement synchronously
for(int i=0; i<SITENUM; i++)
{
    Trig_Point[i] =fovie1.GetMeasResult(i, MVRET, TRIG_RESULT); //Get the position of trigger point for fovie1
    T_delay->SetTestResult(i, 0, (Trig_Point[i]-1)*interval_Meas/1000); //mS
}
fovie0.Set(FV, 0, FOVle_10V, FOVle_100MA, FOVle_RELAY_ON);
fovie1.Set(FV, 0, FOVle_10V, FOVle_1MA, FOVle_RELAY_ON);
fovie0.Set(FV, 0, FOVle_10V, FOVle_100MA, FOVle_RELAY_OFF);
fovie1.Set(FV, 0, FOVle_10V, FOVle_1MA, FOVle_RELAY_OFF);
```

• FOVle –AWG&Digitizer 样例





Make Testing More Valuable